BIT605 Data Structures and Algorithms Assessment 1

Weighting

30%

Learning outcomes

1 Describe fundamental data structures and the operation of abstract data types/elements.
2 Investigate the performance of selected searching algorithms; sequential and binary.
3 Describe various classes of algorithms and their functions and applications.

Instructions

Complete and submit your assessment according to the Open Polytechnic's Assessments webpage. This includes information on academic integrity, formatting, word limits and referencing.

- Include your name, student number and the assessment number.
- Number your pages.

Submission

- Create a word-processed document (MS Word .docx) to complete your written answers to this assessment task. Save this document and all program code within the folder **BIT605_Assessment1_Your Student Number**. Remember to keep your code in separate files, not pasted into your document, and check that your folder doesn't contain material that is not part of this assessment. Archive the folder with a suitable tool, such as 7-zip, to create a single compressed zip file for submission.
- Submit your work in a compressed zip file through your iQualify course.
- Emailed assessments will not be accepted.
- You will receive an automated notice advising you of your successful submission.

By submitting your assessment, you confirm that it is your own, original work.

© The Open Polytechnic of New Zealand

Assessment instructions

You have been hired as a junior programmer by a non-profit organisation called Wiremu ResQ Centre, and you will be developing software solutions for your employer. In this position, you will be collaborating with a small team of developers.

You will interpret a short case study and use it to define a data structure for a simple application. You will then write some program code to store data in these structures and implement two search algorithms. Finally, you will analyse two algorithms to determine the key characteristics and recommend one of them.

Assessment guidelines

This assessment comprises four tasks that provide insights into the interpretation or requirements that lead towards developing data structures and algorithms. **All code must be written in Python unless otherwise specified**.

The four tasks are as follows:

Task 1 Interpret a case study and define a simple data structure.

Task 2 Write code to store purchase orders in both a linked list and a binary tree.
Examine the performance of each algorithm.

Task 3 Write code to find an element in an array using both sequential and binary searches.
Examine the performance of each algorithm.

Task 4 Determine the key characteristics of two algorithms for moving stock.
Based on the key characteristics, justify choosing one of the algorithms.

Using generative AI in your assessment

If you use generative AI (GenAI) for a task instead of your own program code or problem solving, then you must provide:

- the name of the GenAI tool
- the exact prompt used to generate any program code or solve a problem for the task, e.g.,'Create python data-structures for the following information'
- a rationale for the prompt, explaining the details and expected outcome of the prompt
- analysis of the problems (if any) of the codes/data structures/solutions from GenAI
- any variations of the prompt you tried, with explanations as to why you made the changes to the prompt
- indication of the extent you have used AI in your work according to the scale shown in the **Appendix**.

© The Open Polytechnic of New Zealand

Case study: Wiremu ResQ Centre

Wiremu ResQ Centre is a technology recycling centre, a non-profit organisation owned by Wiremu Broughton, with a goal on sustainability. The company takes in old or unwanted computing hardware and other devices to prevent excessive e-waste. They acquire the technology from the public or other companies that deem the technology for disposal. Recycling is enabled by a few permanent employees and volunteers.

By recycling the old technology, ResQ can make refurbished devices and components available to people, schools and other businesses that cannot afford to buy new expensive technology. The devices and such are sold off at a very affordable price to various consumers.

In this assessment, you will be focusing on recording and retrieving purchase orders for the website. Each task supports the next task. For example, the data structures in Task 1 are used in the data manipulation algorithms in Task 2.

Wiremu ResQ Centre has provided a paper-based order form in **AS1_PurchaseOrder.pdf**. This form contains the main information they use for processing mail-order purchases and will form the basis for their web-based process. When you examine the form, feel free to assume the context of the information in it.

Task 1: Defining a data structure

This task involves interpreting requirements and creating data structures that best reflect the requirements. Consider the Wiremu ResQ Centre case study and answer the following questions within the context of the case study.

For this task, you are focusing on the contents of the purchase order form from the case study.

If you are using GenAI for Task 1, then follow the instructions for using GenAI above.

Complete the following:

a. Download a copy of the **AS1_PurchaseOrder.pdf** from iQualify and examine the form. You will notice various places on the form that need to be filled in.

   Examine the form for potential data items. Use suitable names that can be easily translated into data structure(s) for use in program code. There are at least 16 recordable items on the form; this excludes items that can be duplicated and items that can be split off.

© The Open Polytechnic of New Zealand

You must **use at least 10 data items** from the form.

Use the information from the form to create suitable data structures that can hold the above information. The data structures need to be written in Python as classes, with each data item added as an attribute.

You can start with the following as a template:

```
class PurchaseOrder():

    def __init__(self,name="",......):
        self.purchaser_name:str = name # up to 100 characters
```

You should implement at least two classes (for example, `PurchaseOrder`, `OrderLine`, Customer, etc.). Default values and calculations should be included where appropriate.

All classes for this task should be placed in a single file called **purchase.py**, to be submitted for this assessment.

You will make use of this information for Tasks 2 and 3.

(15 marks)

b. Discuss the data structures created in Task 1a. This includes the reason for putting the various fields into each of the chosen data structures. For example, explain why some fields might be not included in your `PurchaseOrder` but inserted into the `OrderLine` data structure.

(5 marks)
(Task 1 total: 20 marks)

© The Open Polytechnic of New Zealand

Task 2: Storing purchase orders

This task demonstrates an understanding of concrete data structures and their implementation and evaluation. Consider the Wiremu ResQ Centre case study and answer the following questions within the context of the case study.

For this task, you will implement two concrete data structures to store multiple `PurchaseOrder` instances from Task 1b.

Note: For Tasks 2a and 2c of this assessment, you cannot use the Python built-in classes (lists, dictionaries, etc.) or other libraries (like collections, etc.). You must implement the structures yourself unless otherwise indicated in the task.

If you are using GenAI for Task 2, then follow the instructions for using GenAI above.

Complete the following:

a. Implement a **linked list** and **binary tree** class to store `PurchaseOrder` instances. The classes must be called `PurchaseOrdersLL` and `PurchaseOrdersBT` and must have a method that adds a `PurchaseOrder` instance to the data structure. The method should use the following signature:
```
add(self, PurchaseOrder,...)
```

   Include code comments describing the various parts of the program code.
   When you have implemented the classes and methods, test that they work. Take a screenshot that documents your test results and add it to the assessment.

   All the classes for Task 2a should be placed in a file called **storage.py**, to be submitted for this assessment.

(15 marks)

b. In Task 2a you implemented two concrete data classes. Now you will need to analyse their performance.
   You will need to complete the following tasks:

   - Examine the runtime performance of the `add()` method on the `PurchaseOrdersLL` and `PurchaseOrdersBT`.
     You can use the partial **AS1_add_perf_template.py** file as a template or generative AI to start your timing code.
   - Download the file **AS1_unsorted_orders.csv** from iQualify to use the data for the testing. It contains about 10,000 records (only sorted by name not number).
   - In your assessment document, record the timings for at least **five runs** on the data structures.

- Store your code in a file called **add_performance.py**, to be submitted for this assessment.
- Example timings output:

```
Run 1
 LinkedList: Added 10000 records in 8.052 milliseconds
 BinaryTree: Added 10000 records in 12.878 milliseconds
Run 2
 LinkedList: Added 10000 records in 6.531 milliseconds
 BinaryTree: Added 10000 records in 13.330 milliseconds
Run 3
 LinkedList: Added 10000 records in 12.242 milliseconds
 BinaryTree: Added 10000 records in 12.395 milliseconds
Run 4
 LinkedList: Added 10000 records in 6.531 milliseconds
 BinaryTree: Added 10000 records in 12.993 milliseconds
Run 5
 LinkedList: Added 10000 records in 6.643 milliseconds
 BinaryTree: Added 10000 records in 19.877 milliseconds
```

(10 marks)

c. Discuss the performance of the two data structures and their theoretical big O.

(Word count guideline for comments: 100–200 words)

(5 marks)
(Task 2 total: 30 marks)

Task 3: Finding a purchase order

This task demonstrates an understanding of search algorithms and their implementation and evaluation. Consider the Wiremu ResQ Centre case study and answer the following questions within the context of the case study.

For this task, you will use the linked list and binary tree data structures from Task 2 as part of a simple search engine. You are expected use the `PurchaseOrder` data structure from Task 1b.

If you are using GenAI for Task 3, then follow the instructions for using GenAI above.

All the classes for Task 3a/b should be placed in a file called storage.py, to be submitted for this assessment.

Complete the following:

a. Implement a **linear search** on the linked list data from Task 2a. You will need to complete the following tasks:

   - Add a method to `PurchaseOrderLL` called `SearchByNumber`. It should accept an argument containing the purchase order number. The method should return the first `PurchaseOrder` instance that matches the number or `None` if there are no matching records.
   - Use the data from the file **AS1_unsorted_orders.csv** from Task 2 for testing.
   - Test the search algorithm with 8837388 and 1. Document your results for those values.

(5 marks)

b. Implement a **binary search** on the binary tree data from Task 2b. You will need to complete the following tasks:
   - Add a method to `PurchaseOrderBT` called `search_by_number`. It should accept an argument containing the purchase order number. The method should return the first `PurchaseOrder` instance that matches the number or `None` if there are no matching records.
   - Use the data from the file **AS1_unsorted_orders.csv** from Task 2 for testing.
   - Test the search algorithm with 3522452
     and 1. Document your results for those values.

(5 marks)

c. In Tasks 3a and 3b, you implemented two different search algorithms on the same or a similar data structure. Now, you will need to analyse the performance of each implementation.
   Complete the following tasks:

- Examine the runtime performance of the search algorithms by including some additional code in your existing code for time measurements. You can use the partial **AS1_search_perf_template.py** file as a template or generative AI to start your timing code. Use the data from the supplied file **AS1_unsorted_orders.csv** from Task 2 for testing.
- Load the data and then perform searches on the following numbers:
  - 14308662
  - 2805996
  - 2273051
  - 555
- In your document, record the timings for five runs on each data structure. Then comment on the time differences between the linear and binary search algorithms. Include a comparison to the theoretical performance from big O notation.
- Example timings:

```
--------- Linked List ---------
Purchase order 14308662 found: 38.400 microseconds
Purchase order 2805996 found: 182.200 microseconds
Purchase order 2273051 found: 278.500 microseconds
Purchase order 555 missing: 287.100 microseconds
--------- Binary Tree ---------
Purchase order 14308662 found: 2.900 microseconds
Purchase order 2805996 found: 1.300 microseconds
Purchase order 2273051 found: 0.900 microseconds
Purchase order 555 missing: 0.800 microseconds
```

- Store your code in a file called **search_performance.py**, to be submitted for this assessment.

(Word count guideline for comments: 200–400 words)

(10 marks)

d. Would the timings from Task 3c be different if `PurchaseOrder.name` (a string) was used in the search instead of the number? Change the above code and test and document the performance and findings of the searches using the name.
In your document, record the timings for five runs on the new search. Then comment on the time differences between the new data type and the previous algorithms. Include a comparison to the theoretical performance from big O notation.

(10 marks)
(Task 3 total: 30 marks)

Task 4: Choosing an algorithm

For the final task of this assessment, you will be looking at two existing algorithms to determine their big O notation complexity.

Wiremu ResQ Centre has a warehouse and sorting sites for their computer hardware. Because of the way the hardware is assembled, they need to be moved between the sites in a set order, but the complexity of this order increases as the number of components in each assembly increases. Therefore, Wiremu ResQ Centre needs to move the stock between the warehouse and sorting sites according to set rules. To reduce the transport costs, they want to minimise the number of moves required.

In your position as a programmer, they want you to analyse two hardware movement algorithms (Algorithm 1 and Algorithm 2) to determine which one they should use. Both algorithms are written in Python. Download the file **AS1_stock_movement.py** from iQualify and examine the code.

Complete the following:

a. 1. What is the big O notation for each function?

   2. Which algorithm do you think would be faster?

   3. Explain why you think it would be faster and the trade-offs it makes to achieve better performance. You may need to add timing code to verify the performance.

(Word count guideline: 100–200 words)

(10 marks)

b. 1. What abstract data type is used in Algorithm 2?

   2. What alternative abstract data type could be used instead?

   3. Explain why you think so and what benefits you think the alternative abstract data type would provide.

(Word count guideline: 100–200 words)

(10 marks)
(Task 4 total: 20 marks)

Store **all** your program code and the word-processed document in a single compressed zip file to submit for Assessment 1.

© The Open Polytechnic of New Zealand

Appendix

The AI Assessment Scale

1	NO AI	The assessment is completed entirely without AI assistance in a controlled environment, ensuring that students rely solely on their existing knowledge, understanding, and skills **You must not use AI at any point during the assessment. You must demonstrate your core skills and knowledge.**
2	AI PLANNING	AI may be used for pre-task activities such as brainstorming, outlining and initial research. This level focuses on the effective use of AI for planning, synthesis, and ideation, but assessments should emphasise the ability to develop and refine these ideas independently. **You may use AI for planning, idea development, and research. Your final submission should show how you have developed and refined these ideas.**
3	AI COLLABORATION	AI may be used to help complete the task, including idea generation, drafting, feedback, and refinement. Students should critically evaluate and modify the AI suggested outputs, demonstrating their understanding. **You may use AI to assist with specific tasks such as drafting text, refining and evaluating your work. You must critically evaluate and modify any AI-generated content you use.**
4	FULL AI	AI may be used to complete any elements of the task, with students directing AI to achieve the assessment goals. Assessments at this level may also require engagement with AI to achieve goals and solve problems. **You may use AI extensively throughout your work either as you wish, or as specifically directed in your assessment. Focus on directing AI to achieve your goals while demonstrating your critical thinking.**
5	AI EXPLORATION	AI is used creatively to enhance problem-solving, generate novel insights, or develop innovative solutions to solve problems. Students and educators co-design assessments to explore unique AI applications within the field of study. **You should use AI creatively to solve the task, potentially co-designing new approaches with your instructor.**

Perkins, Furze, Roe & MacVaugh (2024). The AI Assessment Scale

© Perkins, Furze, Roe & MacVaugh

Source: https://aiassessmentscale.com/

Using the AI assessment scale - https://learnteach.stir.ac.uk/how-to/ai-assessment-scale/

Marking schedule

Task 1a	10–15 marks	7.5–9.5 marks	6–7 marks	1–5.5 marks
Create data structures from sample form.	Presents **proper** data structures that implement two or more classes and includes 10 or more unduplicated items. Data structures are accurately and consistently written with one or no omission from the data items table. Consistently uses names that are easily translated into data structures. Comments are accurate and applicable. Or Zero or few shot prompting was used. Prompt has clear objectives and clarity ensured alignment with desired outcomes. The code generated was **fully** functional and met **all** requirements. Explanation of the code and analysis of the problems (if any) were **clear** and **accurate**.	Presents **adequate** data structures that implement two or more classes and includes 10 or more unduplicated items. A few errors or inconsistencies are evident, but the overall integrity of the response is maintained. Or Zero or few shot prompting was used. Prompt is **mostly** clear with objectives ensuring alignment with desired outcomes. The code generated was functional and met **most** requirements. Explanation of the code and analysis of the problems (if any) were **mostly clea**r and **accurate**.	Presents **partial** data structures that implement two or more classes with some item omissions or duplications may be evident. Several errors or omissions are evident in the data structures or data items. Or Zero-shot prompting was used. Prompt had **some** objectives aligning with the desired outcomes. The code generated was mostly functional and met **some** requirements. Explanation of the code and analysis of the problems (if any) were **brief** and **mostly accurate**.	Presents some data structures but with recurring gaps in data information or persistent structural errors. There may be several significant omissions or duplications in the items. Or Zero-shot prompting was **attempted**. Prompt had **some** objectives aligning with the desired outcomes. The code generated was **not** functional. The code met **none or few** requirements. Explanation of the code and analysis of the problems (if any) were not **accurate**.
Task 1b	**3.5–5 marks**	**2.5–3 marks**	**2 marks**	**1–1.5 marks**

© The Open Polytechnic of New Zealand

Discuss the data structures created in Task 1a.	Presents data structures with **all** separations of concerns between the two data structures. Reasons **properly** support location of each field per data structure.	Presents data structures with **most** separations of concerns between the two data structures. Reasons **adequately** support location of each field per data structure. There may be one or two omissions.	Presents data structures with **some** separations of concerns between the two data structures. Reasons **partially** support location of each field per data structure. There are several omissions.	Presents a data structure with **almost no** separations of concerns. Reasons **do not or very poorly** support the location of each field in the data structure. There are many significant omissions.
Task 2a	**10-15 marks**	**7.5–9.5 marks**	**6–7 marks**	**1–5.5 marks**
Implement a linked list and binary tree class and an add method for PurchaseOrder instances. Test that they work.	Has **properly** implemented both binary tree class and linked list class as well as a method for PurchaseOrder instances. Testing shows that the implementation works. Coding is accurate and fully functional. Or Zero or few shot prompting has been used. Prompt has clear objectives and clarity ensured alignment with desired outcomes. The code generated is **fully** functional and meets **all** requirements. Explanation of the code and analysis of the problems (if any) are **clear** and **accurate**.	Has **adequately** implemented both linked list class and binary tree class as well as a method for PurchaseOrder instances. One or two errors or inconsistencies are evident, but the overall integrity of the response is maintained. Testing is documented, though the error/s or inconsistencies may impact the results. Or Zero or few shot prompting has been used. Prompt is **mostly** clear with objectives ensuring alignment with desired outcomes. The code generated was functional and meets **most** requirements. Explanation of the code and analysis of the	Has **partially** implemented this task's requirement for two classes and a method for PurchaseOrder instances. Several errors or omissions from task requirements are evident in the coding. Testing may be inadequate or show that implementation is not fully functional. Or Zero-shot prompting was used. Prompt had **some** objectives aligning with the desired outcomes. The code generated is mostly functional and meets **some** requirements. Explanation of the code and analysis of the problems (if any) are	Shows some evidence of implementation of either linked list class or binary tree class and method for PurchaseOrder instances but with persistent coding errors. Class and/or method are dysfunctional. Or Zero-shot prompting has been **attempted**. Prompt has **some** objectives aligning with the desired outcomes. The code generated is **not** functional. The code meets **none or few** requirements. Explanation of the code and analysis of the problems (if any) are not **accurate**.

© The Open Polytechnic of New Zealand

		problems (if any) are **mostly clea**r and **accurate**.	**brief** and **mostly accurate**.	
Task 2b	**6.5–10 marks**	**5–6 marks**	**4–4.5 marks**	**1–3.5 marks**
Provide an analysis of the performance for Task 2a	Has added **proper** code to test runtime performance. Documents **all** timings for five runs on each data structure.	Has added **adequate** code to test runtime performance. Documents **most** timings for five runs on each data structure.	Has added **partial** codes to test runtime performance. Provides **some** run timing information for the data structures.	Attempts to add codes to test runtime performance. However, they are **incomplete** or **inaccurate**. Run timing information may show errors or be incomplete.
Task 2c	**3.5–5 marks**	**2.5–3 marks**	**2 marks**	**1–1.5 marks**
Discuss the performance of the two data structures and their theoretical big O.	Provides clear and thorough analysis of time differences between structures and comparing big O notation. Coding, timings and analysis are accurate and applicable.	Provides clear analysis of time differences between structures and comparing big O notation. One or two errors or inconsistencies are evident in coding, timings or analysis, but overall integrity of the response is maintained.	Provides some commentary on time differences between structures and comparing big O notation. Several errors or omissions are evident in coding, timings or analysis.	Provides some comparative commentary. Persistent functional errors are evident in the code and/or the commentary is significantly flawed or incomplete.
Task 3a	**3.5–5 marks**	**2.5–3 marks**	**2 marks**	**1–1.5 marks**
Add linear search method on linked list data and test the search algorithm.	Has **properly** implemented linear search method and tested existing/non-existing values. Testing shows that the implementation works. Coding is accurate and fully functional.	Has **adequately** implemented linear search method and tested existing/non-existing values. One or two errors or inconsistencies are evident, but the overall integrity of the response is maintained. Testing is documented, though the	Has **partially** implemented linear search method and tested existing/non-existing values. Several errors or omissions from task requirements are evident in the coding. Testing may be inadequate or show that	Has implemented some aspects of a linear search method but with persistent functional errors. Algorithm may be incapable of accepting some argument or value types.

© The Open Polytechnic of New Zealand

	Or Zero or few shot prompting has been used. Prompt has clear objectives and clarity ensuring alignment with desired outcomes. The code generated is **fully** functional and meets **all** requirements. Explanation of the code and analysis of the problems (if any) are **clear** and **accurate**.	error/s or inconsistencies may impact the results. Or Zero or few shot prompting is used. Prompt is **mostly** clear with objectives ensuring alignment with desired outcomes. The code generated is functional and meets **most** requirements. Explanation of the code and analysis of the problems (if any) are **mostly clea**r and **accurate**.	implementation is not fully functional. Or Zero-shot prompting is used. Prompt has **some** objectives aligning with the desired outcomes. The code generated is mostly functional and meets **some** requirements. Explanation of the code and analysis of the problems (if any) are **brief** and **mostly accurate**.	Or Zero-shot prompting is **attempted**. Prompt has **some** objectives aligning with the desired outcomes. The code generated is **not** functional. The code meets **none or few** requirements. Explanation of the code and analysis of the problems (if any) are not **accurate**.
Task 3b	**3.5–5 marks**	**2.5–3 marks**	**2 marks**	**1–1.5 marks**
Add binary search method on binary tree data and test the search algorithm.	Has **properly** implemented binary search method and tested existing/non-existing values. Testing shows that the implementation works. Coding is accurate and fully functional. Or Zero or few shot prompting is used. Prompt has clear objectives and clarity ensuring alignment with desired outcomes. The code generated is **fully**	Has **adequately** implemented binary search method and tested existing/non-existing values. One or two errors or inconsistencies are evident, but the overall integrity of the response is maintained. Testing is documented, though the error/s or inconsistencies may impact the results. Or Zero or few shot prompting is used. Prompt is **mostly** clear	Has **partially** implemented binary search method and tested existing/non-existing values. Several errors or omissions from task requirements are evident in the coding. Testing may be inadequate or show that implementation is not fully functional. Or Zero-shot prompting is used. Prompt had **some** objectives aligning with the desired outcomes.	Has implemented some aspects of a binary search method but with errors leading to persistent dysfunction. Algorithm may be incapable of accepting some argument or value types. Or Zero-shot prompting is **attempted**. Prompt has **some** objectives aligning with the desired outcomes. The code generated is **not** functional. The code

© The Open Polytechnic of New Zealand

	functional and meets **all** requirements. Explanation of the code and analysis of the problems (if any) are **clear** and **accurate**.	with objectives ensuring alignment with desired outcomes. The code generated is functional and meets **most** requirements. Explanation of the code and analysis of the problems (if any) are **mostly clea**r and **accurate**.	The code generated is mostly functional and meets **some** requirements. Explanation of the code and analysis of the problems (if any) are **brief** and **mostly accurate**.	meets **none or few** requirements. Explanation of the code and analysis of the problems (if any) are not **accurate**.
Task 3c	**6.5–10 marks**	**5–6 marks**	**4–4.5 marks**	**1–3.5 marks**
Provide an analysis of the performance for Task 3a and Task 3b with numbers as test data.	Has added **proper** codes to test runtime performance and used specified searches. Documents **all** timings for five runs on each data structure. Provides clear and thorough analysis of time differences between search algorithms and comparing big O notation. Coding, timings and analysis are accurate and applicable.	Has added **adequate** codes to test runtime performance and used specified searches. Documents **most** timings for five runs on each data structure. Provides clear analysis of time differences between search algorithms and comparing big O notation. One or two errors or inconsistencies are evident in coding, timings or analysis, but the overall integrity of the response is maintained.	Has added **partial** codes to test runtime performance and used specified searches. Provides **some** run timing information for the data structures. Provides some commentary on time differences between search algorithms and comparing big O notation. Several errors or omissions are evident in coding, timings or analysis.	Attempts to add codes to test runtime performance and uses specified searches. However, they are **incomplete** or **inaccurate**. Run timing information may show errors or be incomplete. Provides some comparative commentary. Persistent functional errors are evident in the code or timing information, or the commentary is significantly flawed or incomplete.
Task 3d	**6.5–10 marks**	**5–6 marks**	**4–4.5 marks**	**1–3.5 marks**
Provide an analysis of the performance for Task 3a and Task 3b with text as test data.	Has added **proper** codes to test the updated search.	Has added **adequate** codes to test runtime performance and used specified searches.	Has added **partial** codes to test runtime performance and used specified searches.	Has **attempted** to add code to test runtime performance and used specified searches.

© The Open Polytechnic of New Zealand

	Documents **all** timings for five runs on each data structure. Provides **clear** and **thorough** analysis of time differences between the new search algorithm and the algorithms from Task 3a and 3b. Coding, timings and analysis are accurate and applicable.	Documents **most** timings for five runs on each data structure. Provides **some** analysis of the time differences between search algorithms and comparing big O notation. One or two errors or inconsistencies are evident in coding, timings or analysis, but the overall integrity of the response is maintained.	Provides **some** run-timing information for the data structures with some errors and inconsistencies. Provides **some** commentary on time differences between search algorithms and comparing big O notation. Several errors or omissions are evident in coding, timings or analysis.	However, they are **incomplete** or **inaccurate**. Run timing information may show errors or be incomplete. Attempted to provide some comparative commentary. Persistent functional errors are evident in the code or timing information, or the commentary is significantly flawed or incomplete.
Task 4a	**6.5–10 marks**	**5–6 marks**	**4–4.5 marks**	**1–3.5 marks**
1. Identify big O notation for each function. 2. Identify the faster algorithm. 3. Explain the choice of faster algorithm and its trade-offs.	1. **Correctly** identifies big O notation for each function. 2. **Correctly** identifies faster algorithm. 3. Offers an accurate and well-supported explanation for choice of faster algorithm and its trade-offs.	1. **Correctly** identifies big O notation for each function. 2. **Correctly** identifies faster algorithm. 3. Offers a clear explanation for choice of faster algorithm and its trade-offs. One or two errors or inconsistencies are evident, but the overall integrity of the response is maintained.	**Incorrectly** identifies **EITHER** 1. big O notation for each function **OR** 2. faster algorithm. 3. Offers explanation for choice of faster algorithm and its trade-offs. Several inaccuracies or omissions are evident.	1. Attempts to identify big O notation for each function though errors may be evident. 2. Identification of faster algorithm may not be fully accurate. 3. Provides some discussion on choice of faster algorithm but with errors or omissions that invalidate some of the response.
Task 4b	**6.5–10 marks**	**5–6 marks**	**4–4.5 marks**	**1–3.5 marks**
1. Identify abstract data type being used.	1. **Correctly** identifies abstract data type being used.	1. **Correctly** identifies abstract data type being used.	1. **Partially i**dentifies abstract data type being used.	1. Identifies an abstract data type though may not be fully correct.

© The Open Polytechnic of New Zealand

2. Suggest alternative abstract data type. 3. Explain choice of alternative abstract data type and its benefits.	2. **Properly** suggests an alternative abstract data type. 3. Offers an accurate and well-justified explanation for suggestion of alternative abstract data type, fully addressing its benefits.	2. **Adequately** suggests an alternative abstract data type with one or two minor omissions. 3. Supports the suggestion of an alternative abstract data type with clear explanations of its benefits. One or two errors or inconsistencies are evident, but the overall integrity of the response is maintained.	2. Suggests an alternative abstract data type with several omissions. 3. Offers some explanation for choice of alternative abstract data type and its benefits. Several inaccuracies or omissions are evident in the response.	2. Suggests an alternative abstract data type that may not be applicable. 3. Offers some commentary on the choice of alternative abstract data type, though its benefits may not be addressed. Significant errors or omissions are evident that invalidate some of the response.

© The Open Polytechnic of New Zealand